Selection via Proxy

https://openreview.net/pdf?id=HJg2b0VYDr

Главная идея: для отбора данных для трейна не обязательно каждый раз обучать большую модель. Достаточно дешевой proxy-модели, которая хорошо ранжирует примеры по полезности.

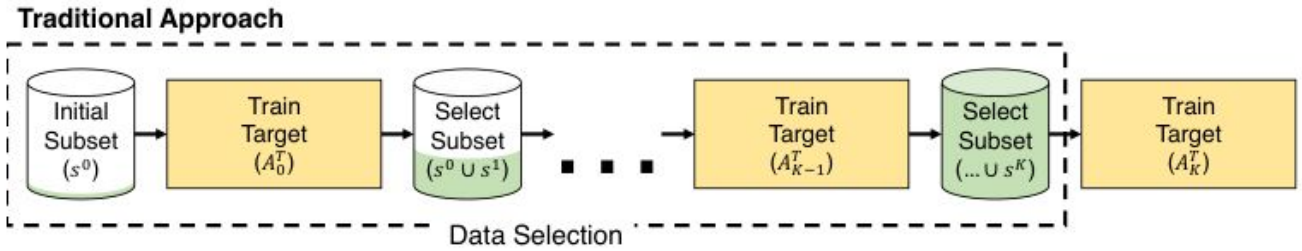
Proxy-модель выбирает данные -> target обучается на выбранном наборе

Проблема: выбор данных сам становится дорогим

Многие методы отбора данных в глубоком обучении требуют сигнала от уже обученной модели: вероятностей классов или осмысленного представления признаков.

Существующие методы

- uncertainty sampling требует вероятности классов: насколько модель сомневается
- k-centers требует feature representation: где объекты находятся в embedding-пространстве
- forgetting events требуют наблюдать, какие примеры модель забывает во время обучения
- в active learning цикл повторяется после каждой новой партии размеченных данных: обучить модель → выбрать следующую партию → разметить → переобучить модель

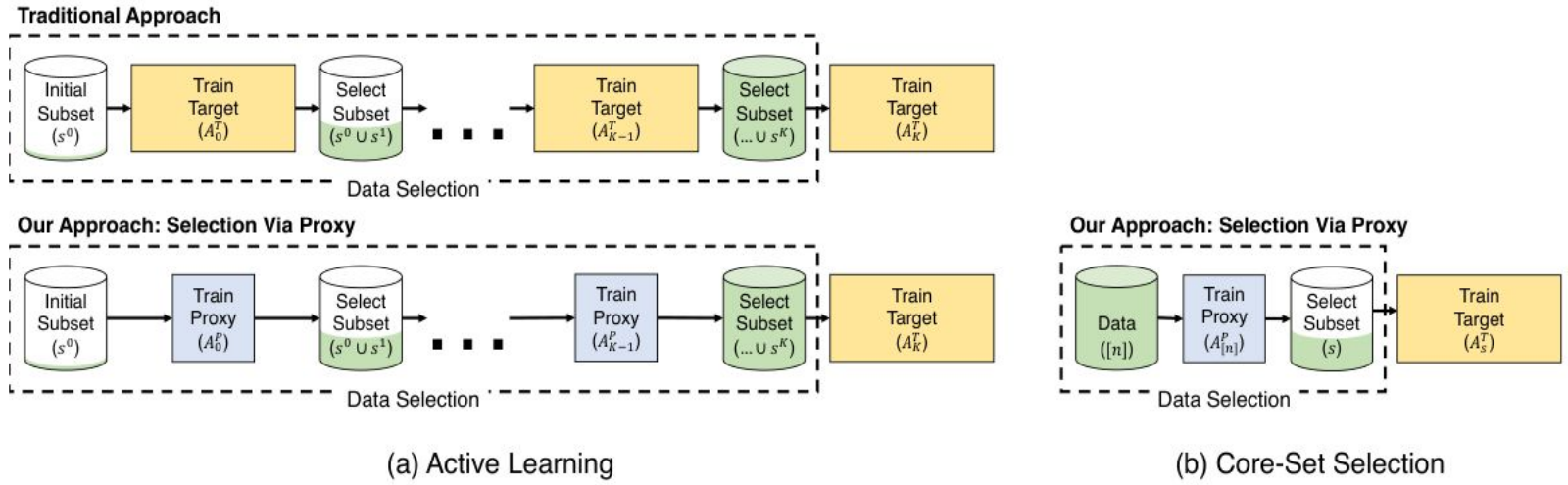


Парадокс: хотим сократить затраты на разметку, но сам выбор данных требует многократного обучения модели

Идея SVP: proxy-модель делает отбор, target — финальное обучение

Авторы предлагают ускорить существующие методы выбора данных с помощью proxy-модели. Их метод разделяет две роли, которые обычно выполняет одна большая модель.

- proxy model: маленькая, быстрая, может быть менее точной, ее задача — не финальная точность, а ранжирование примеров
- target model: большая и точная модель, которую хотим получить в итоге, она обучается после того, как proxy уже помогла выбрать данные



Идея: хорошая proxy-модель должна выбирать похожие примеры, а не быть лучшим классификатором

Как понять, какие данные стоит выбрать?

В статье рассматриваются уже существующие методы отбора данных. Они по-разному определяют, какие примеры наиболее полезны для обучения.

- **Uncertainty sampling — выбрать то, в чем модель сомневается**
  Например: cat 52%/dog 48% → модель не уверена → такой пример стоит добавить в разметку
  Но нужны вероятности классов от уже обученной модели
- **Greedy k-centers — выбрать максимально разные примеры**
  Модель переводит каждый объект в эмбединг, затем выбираются точки, которые находятся далеко от уже выбранных и лучше покрывают пространство данных.
  Нужны осмысленные эмбединги из hidden layers обученной модели

Algorithm 1 GREEDY K-CENTERS (WOLF, 2011; SENER & SAVARESE, 2018)

Input: data $\mathbf{x}_i$, existing pool s^0, trained model A_0^T, and a budget b

Initialize $s = s^0$
repeat
  $u = \arg\max_{i \in [n] \setminus s} \min_{j \in s} \Delta\left(\mathbf{x}_i, \mathbf{x}_j; A_0^T\right)$
  $s = s \cup \{u\}$
until $|s| = b + |s^0|$
return $s \setminus s^0$

Как понять, какие данные стоит выбрать?

В статье рассматриваются уже существующие методы отбора данных. Они по-разному определяют, какие примеры наиболее полезны для обучения.

- **Forgetting events — оставить трудные примеры**
  Во время обучения пример может сначала классифицироваться правильно, а позже снова неправильно: правильно → правильно → ошибка = 1 forgetting event.
  Чем чаще пример “забывается”, тем он считается более трудным и информативным.
  Нужно отслеживать поведение модели на протяжении обучения

Algorithm 2 FORGETTING EVENTS (TONEVA ET AL., 2019)

```
Initialize prev_acc_i = 0, i ∈ [n]
Initialize forgetting_events_i = 0, i ∈ [n]
while training is not done do
    Sample mini-batch B from L
    for example i ∈ B do
        Compute acc_i
        if prev_acc_i > acc_i then
            forgetting_events_i += 1
        prev_acc_i = acc_i
    Gradient update classifier on B
return forgetting_events
```

Проблема: все методы требуют сначала обучить модель, чтобы получить сигнал для выбора данных
Идея SVP: **этот сигнал получать от маленькой дешевой proxy-модели, а большую target-модель обучать только на уже выбранных данных**

Как авторы делают proxy: меньше модель или меньше эпох

Proxy нужна не для финального предсказания — она нужна только для выбора данных. Поэтому авторы специально делают ее проще и быстрее, чем target-модель.

Есть два способа, как сделать proxy дешевле:

1. Взять меньшую модель: она хуже по точности, но обучается гораздо быстрее

- CIFAR10 / CIFAR100: **ResNet164 → ResNet20/56/110**
- ImageNet: **ResNet50 → ResNet18**
- Amazon Reviews: **VDCNN29 → VDCNN9 или fastText**

2. Обучать proxy меньше эпох: авторы проверяют, можно ли получить достаточно хороший сигнал для отбора еще до полной сходимости модели

Dataset	Model	Number of Parameters (millions)
CIFAR10	ResNet164	1.7
	ResNet110	1.73
	ResNet56	0.86
	ResNet20	0.27
	ResNet14	0.18
	ResNet8	0.08
CIFAR100	ResNet164	1.73
	ResNet110	1.74
	ResNet56	0.86
	ResNet20	0.28
	ResNet14	0.18
	ResNet8	0.08

Ключевая идея: для выбора данных нужна не максимальная accuracy proxy, а хорошее ранжирование примеров

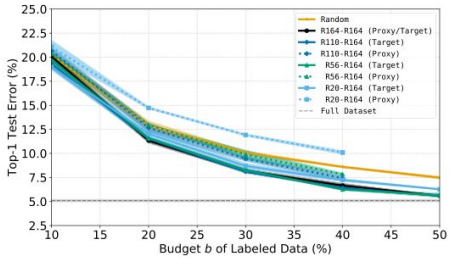
Active learning: proxy выбирает данные почти так же хорошо

Сначала размечено только 2% данных. Затем модель несколько раз выбирает новые batch’и для разметки.

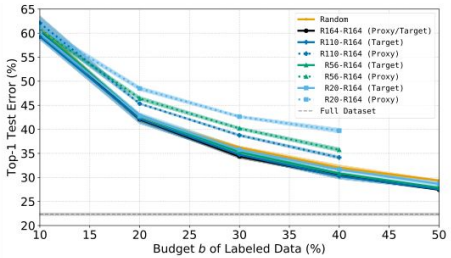
- пунктирные линии — качество самой proxy: оно часто хуже
- сплошные линии — качество target после данных, выбранных proxy

Если сплошные линии близки к baseline, значит proxy выбрала хорошие данные. На CIFAR и ImageNet target почти не страдает от замены selection-модели.

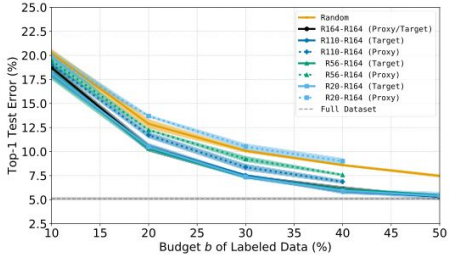
То, что proxy-модель слабая как классификатор не означает, что она плохая proxy для выбора данных



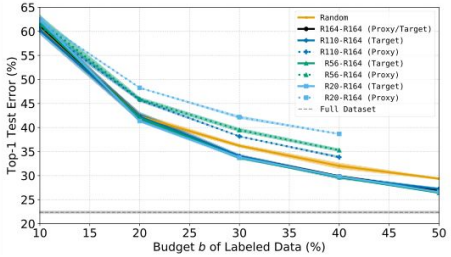
(a) CIFAR10 greedy k-centers



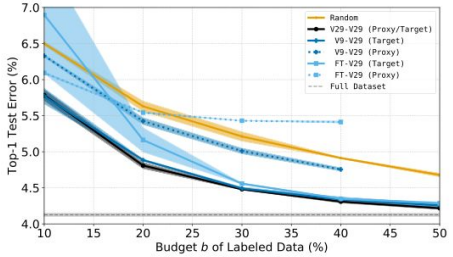
(b) CIFAR100 greedy k-centers



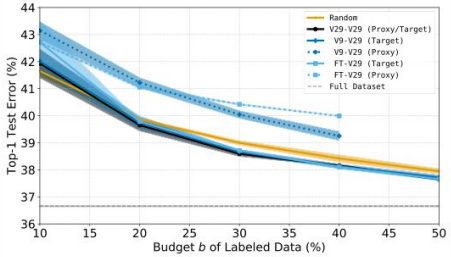
(c) CIFAR10 least confidence



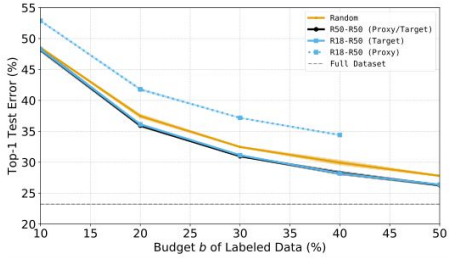
(d) CIFAR100 least confidence



(e) Amazon Review Polarity least confidence



(f) Amazon Review Full least confidence



(g) ImageNet least confidence

Active learning: насколько ускоряется выбор

		Data Selection Speed-up				
	Budget (b/n)	10%	20%	30%	40%	50%
Dataset	**Selection Model**					
CIFAR10	ResNet164 (Baseline)	**1.0**×	**1.0**×	**1.0**×	**1.0**×	**1.0**×
	ResNet110	**1.8**×	**1.9**×	**1.9**×	**1.8**×	**1.8**×
	ResNet56	**2.6**×	**2.9**×	**3.0**×	**3.1**×	3.1×
	ResNet20	**3.8**×	**5.8**×	**6.7**×	**7.0**×	7.2×
CIFAR100	ResNet164 (Baseline)	**1.0**×	**1.0**×	**1.0**×	**1.0**×	**1.0**×
	ResNet110	**1.5**×	**1.6**×	**1.6**×	**1.6**×	1.6×
	ResNet56	**2.4**×	**2.7**×	**3.0**×	**2.9**×	**3.1**×
	ResNet20	4.0×	**5.8**×	**6.6**×	**7.0**×	**7.2**×
ImageNet	ResNet50 (Baseline)	**1.0**×	**1.0**×	**1.0**×	**1.0**×	**1.0**×
	ResNet18	**1.2**×	**1.3**×	**1.4**×	**1.5**×	**1.6**×
Amazon	VDCNN29 (Baseline)	**1.0**×	**1.0**×	**1.0**×	**1.0**×	**1.0**×
Review	VDCNN9	**1.9**×	1.8×	**1.8**×	**1.8**×	1.8×
Polarity	fastText	10.6×	20.6×	32.2×	**41.9**×	51.3×

до x7

на CIFAR10/CIFAR100, когда ResNet20 выбирает данные для ResNet164

до x41.9

на Amazon Review Polarity, когда fastText используется как proxy для VDCNN29

это ускорение этапа отбора данных, а не всего процесса обучения target-модели

Core-set selection: выбрать подмножество и обучить target на нем

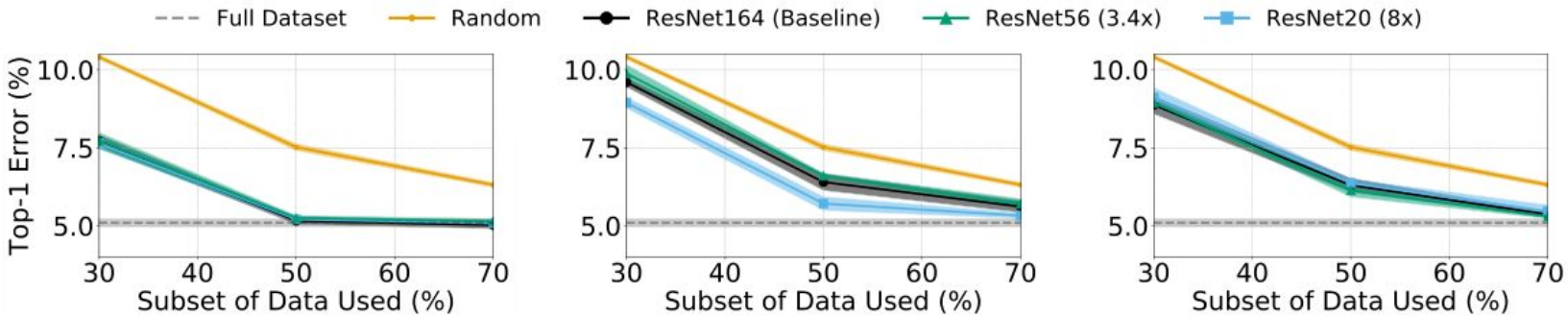
Теперь все данные уже размечены, но обучать target на всем наборе дорого:

- нужно выбрать подмножество данных, которое почти заменяет полный датасет
- proxy обучается на данных и оценивает важность примеров
- target обучается только на выбранном наборе

Если этот набор хороший, получаем меньше время обучения без заметной потери точности. **Это уже экономия не разметки, а времени обучения большой модели.**

На CIFAR10 половину данных можно убрать:

- ResNet20 выбирает подмножество для ResNet164. При forgetting events половина CIFAR10 удаляется без заметного ухудшения качества
- Ускорение в 1.6 раз: учитывается и обучение proxy, и обучение target на уменьшенном наборе



(a) CIFAR10 forgetting events (b) CIFAR10 entropy (c) CIFAR10 greedy k-centers

Почему это работает: proxy и target похоже ранжируют данные

Авторы проверяют не только accuracy, а именно порядок примеров: какие объекты модель считает сложными, неопределенными или репрезентативными.

Например, обе модели могут дать такой порядок:

- **Proxy:** A → C → B → D
- **Target:** A → C → D → B

То есть они могут ошибаться по-разному, но все равно **похоже ранжировать данные по сложности/неопределенности/репрезентативности**.

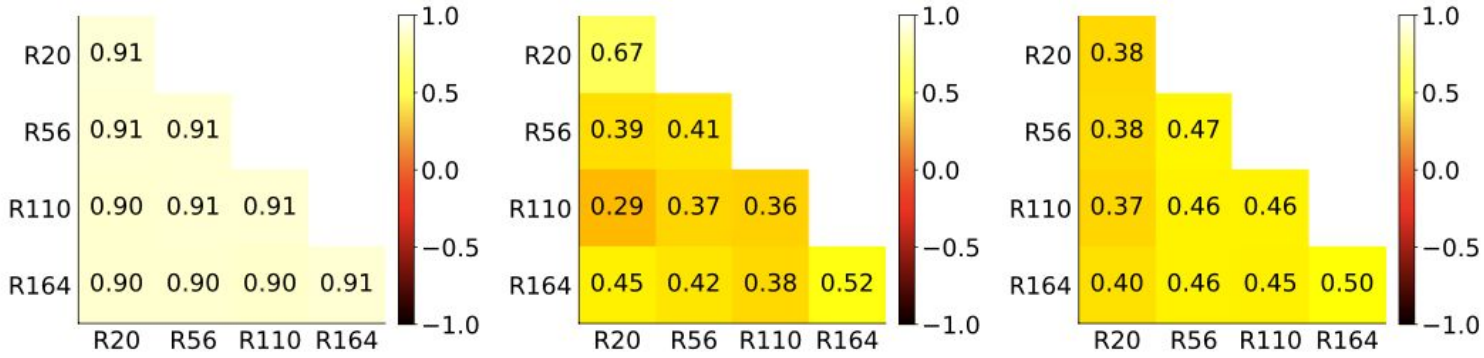
Почему это работает: proxy и target похоже ранжируют данные

Что делают авторы:

- ранжируют все примеры для proxy и target
- сравнивают эти ранки с помощью **Spearman correlation**
- высокая корреляция → модели в целом согласны, какие примеры важнее

Что получилось:

- маленькие ResNet часто ранжируют данные похоже на ResNet164
- для forgetting events совпадение особенно высокое
- иногда различие между ResNet20 и ResNet164 не намного больше, чем различие между двумя независимыми запусками одной и той же ResNet

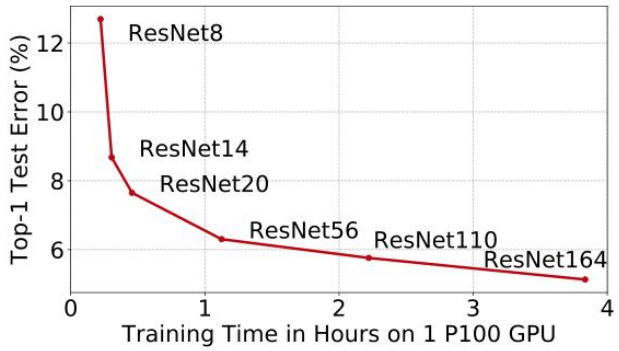


(a) CIFAR10 forgetting events (b) CIFAR10 entropy (c) CIFAR10 greedy k-centers

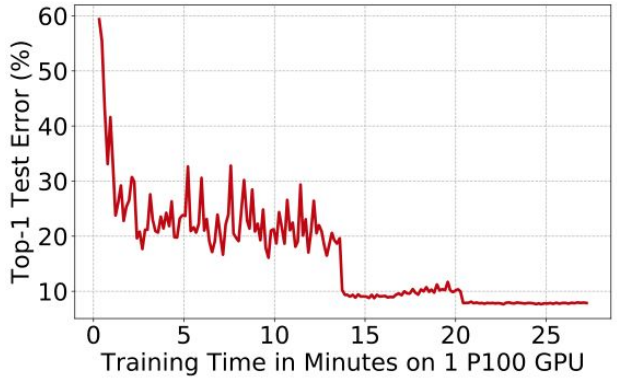
Figure 3: **Comparing selection across model sizes and methods on CIFAR10.** Average Spearman's correlation between different runs of ResNet (R) models and at varying depths. We computed rankings based on forgetting events (left), entropy (middle), and greedy k-centers (right). We saw a similarly high correlation across model architectures (*off-diagonal*) as between different runs of the same architecture (*on-diagonal*), suggesting that small models are good proxies for data selection.

Почему proxy экономит время

Большая модель дает лучшее качество, но последние проценты accuracy стоят дорого, но для выбора не всегда нужна полностью дообученная модель, часто достаточно ранней картины decision boundary и representation → поэтому proxy можно делать меньше или останавливать раньше



(a) Top-1 test error and training time on CIFAR10 for ResNet with pre-activation and a varying number of layers. There is a diminishing return in accuracy by increasing the number of layers.



(b) Top-1 test error during training of ResNet20 with pre-activation. In the first 14 minutes, ResNet20 reaches 9.0% top-1 error, while the remaining 12 minutes are spent on decreasing error to 7.6%

ResNet20: ~26-30 мин

ResNet164: ~4 часа

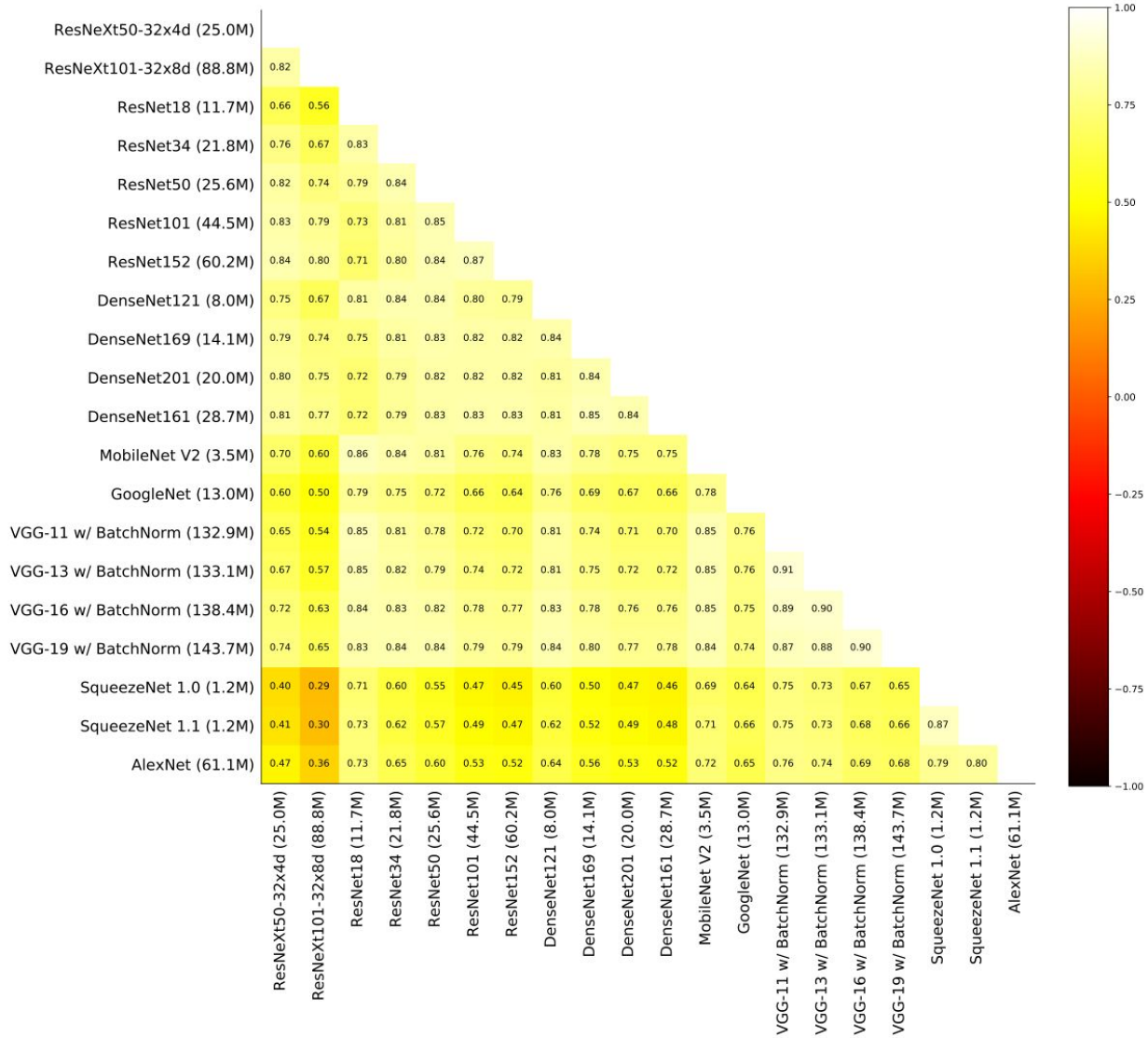
Идея не в том, что ResNet20 лучше, а в том, что для отбора данных ее сигнала часто достаточно.

Proxy не обязана быть той же архитектурой

Авторы сравнивают rankings на ImageNet для разных pretrained-моделей: ResNet, DenseNet, MobileNet, VGG, SqueezeNet и других.

- корреляции остаются заметными даже между разными архитектурами
- MobileNet V2 намного меньше ResNet50, но entropy-ranking похож
- на Amazon fastText тоже может быть proxy для VDCNN

Это показывает, что сигнал для отбора данных может сохраняться даже при смене архитектуры модели



Ограничения и вывод

Ограничения

- SVP не улучшает сам способ отбора данных, а делает уже существующий метод быстрее за счет более дешевой прокси-модели
- Если исходный метод отбора работает плохо, SVP это не исправит. Например, если отбор с большой моделью не лучше случайной выборки, прокси-модель тоже не даст заметного выигрыша
- Стоимость самого алгоритма отбора остается. Например, greedy k-centers требует большого числа вычислений и на ImageNet оказывается слишком дорогим даже при использовании прокси-модели
- На CIFAR100 ранжирование стабилизировалось медленнее, поэтому слишком раннее завершение обучения proxy чаще ухудшало отбор
- Результат зависит от задачи: на CIFAR10 удалось удалить до 50% данных без значимой потери качества; на ImageNet такое сокращение датасета не удалось; на Amazon Review Full даже исходный метод отбора не превосходил случайный отбор

			Top-1 Error (%)			
		Subset Size	40%	60%	80%	100%
Dataset	**Method**	**Selection Model**				
ImageNet	Random	-	32.2 ± 0.12	28.0 ± 0.15	25.8 ± 0.06	23.3 ± 0.11
	Entropy	ResNet50 (Baseline)	34.9 ± 0.08	28.8 ± 0.03	25.9 ± 0.04	-
	Entropy	ResNet18	32.2 ± 0.04	27.0 ± 0.01	25.1 ± 0.07	-
	Forgetting Events	ResNet50 (Baseline)	31.9 ± 0.07	26.7 ± 0.06	24.8 ± 0.03	-
	Forgetting Events	ResNet18	31.6 ± 0.07	27.1 ± 0.10	25.3 ± 0.18	-
Amazon Review Polarity	Random	-	4.9 ± 0.02	4.5 ± 0.05	4.3 ± 0.01	4.1 ± 0.04
	Entropy	VDCNN29 (Baseline)	4.4 ± 0.03	4.2 ± 0.02	4.2 ± 0.02	-
	Entropy	VDCNN9	4.4 ± 0.02	4.2 ± 0.01	4.2 ± 0.00	-
	Entropy	fastText	4.4 ± 0.02	4.2 ± 0.02	4.2 ± 0.02	-
Amazon Review Full	Random	-	38.4 ± 0.03	37.6 ± 0.03	37.0 ± 0.05	36.6 ± 0.06
	Entropy	VDCNN29 (Baseline)	42.7 ± 1.14	39.3 ± 0.14	37.6 ± 0.10	-
	Entropy	VDCNN9	41.1 ± 0.24	38.8 ± 0.03	37.7 ± 0.09	-
	Entropy	fastText	39.0 ± 0.18	37.8 ± 0.06	37.1 ± 0.06	-

Главный вывод

Для выбора данных не всегда нужна большая и дорогая целевая модель. Если небольшая прокси-модель похожим образом определяет, какие примеры важные, сложные или неопределенные, ее можно использовать для отбора данных значительно дешевле.